코드 리뷰 브리핑

멀티미디어 아카이브 서비스 DDD 관점 리뷰

main.py · domain/archive.py · schemas/archive.py

한 줄 결론

지금은 DDD라기보다 “똑똑한 DTO + 두꺼운 컨트롤러”

엔티티에 상태 전이를 넣은 출발점은 좋지만, 규칙은 main.py에 흩어져 있고 불변식은 우회할 수 있습니다.

MAIN.PY 들여다보기

main.py의 코드가 전부 문제는 아닙니다

여기 있어도 되는 것

표현 계층의 관심사

- HTTP 상태 코드 매핑
- Content-Disposition 등 응답 헤더
- StreamingResponse 스트리밍
- 요청 파싱(UploadFile)

옮겨야 하는 것

도메인 규칙과 유스케이스 흐름

- 파일명 정제, MIME 기본값
- ID 발급과 저장 경로 규칙
- 변환 파이프라인 상태 조율
- 실패 처리(로그 한 줄로 덮음)

흩어진 규칙

main.py에 숨어 있는 판단 7가지

main.py의 코드	성격	가야 할 곳
f"seq_{id}_{filename}"	저장 규칙 (두 곳에 중복)	인프라 FileStorage
os.path.basename(...)	파일명 정제 규칙	값 객체 FileName
content_type or "octet-stream"	MIME 기본값 정책	값 객체 ContentType
asset_id_sequence 전역 변수	ID 발급	Repository.next_id()
asset_repository: Dict	영속화	도메인 포트 + 인프라 구현
execute_media_transcoding_…	상태 전이 조율	애플리케이션 서비스
except ValueError → 로그만	실패 처리 누락	mark_as_failed()

빠진 상태 전이

변환이 실패하면 자산은 영원히 PROCESSING에 멈춥니다



지금: FAILED 상태가 없음

except ValueError가 로그만 남기고 끝나서, 클라이언트는 PROCESSING만 계속 보게 됩니다.

바꿀 방향: mark_as_failed()

“변환은 끝나거나 실패한다”는 규칙을 엔티티가 직접 표현하게 합니다.

MULTIMEDIAASSET

엔티티의 불변식을 우회하는 길이 열려 있습니다

생성자로 status를 받음

status="COMPLETED"로 만들면 전이 규칙을 건너뜁니다. 새 자산은 항상 PENDING이어야 합니다.

속성이 전부 public

asset.status = "아무거나"가 막히지 않습니다. 상태가 문자열이라 오타도 못 잡습니다.

to_dict()가 도메인에

직렬화는 표현 계층의 일입니다. 스키마의 from_attributes로 변환하면 됩니다.

datetime.now() 직접 호출

타임존이 없고 테스트하기 어렵습니다. UTC를 쓰거나 시각을 주입받습니다.

예외가 전부 ValueError

API가 “잘못된 입력”과 “서버 오류”를 구분하지 못합니다.

해결 방향

AssetStatus Enum, 읽기 전용 프로퍼티, 복원용 팩토리, 도메인 전용 예외

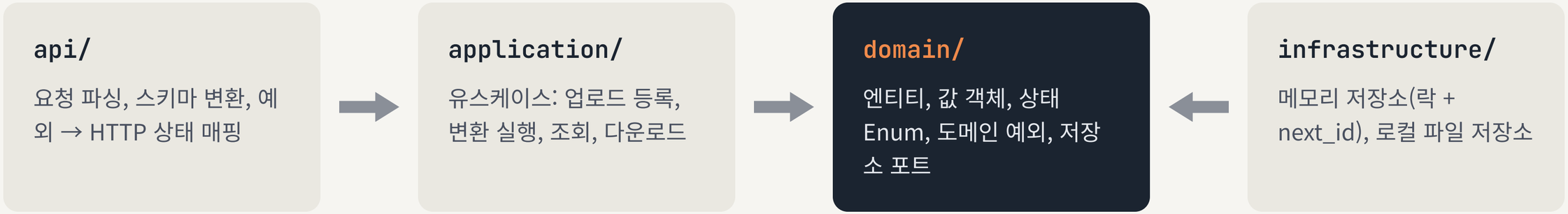
실제 버그

구조 문제는 이미 버그로 나타나고 있습니다

상황	무슨 일이 생기나	고칠 방향
동시 업로드	await 사이에 시퀀스가 안 올라가 같은 ID·같은 파일 경로를 받아 덮어씀	저장소가 ID를 맨 먼저 발급
0바이트 파일	도메인 ValueError가 except Exception에 잡혀 500 응답	도메인 예외 → 400/422
filename 없음	os.path.basename(None)에서 TypeError → 500 응답	FileName 값 객체에서 검증
없는 ID 조회	410 Gone을 반환 (존재한 적 없는 ID)	404 Not Found
변환 중 다운로드	PROCESSING 상태에서도 다운로드 가능	정책 결정 후 엔티티가 판단

권장 구조

의존성은 모두 도메인을 향합니다



저장 경로 규칙 seq_{id}_{name}은 infrastructure 한 곳에만 둡니다. 업로드 핸들러는 다섯 줄 남짓으로 줄어듭니다.

엔티티 스케치

상태는 엔티티만 바꿀 수 있게

- 새 자산은 항상 PENDING
- status는 읽기 전용
- 전이는 _transition 한 곳에서 검증
- FAILED로 실패를 표현

```
class AssetStatus(str, Enum):
    PENDING / PROCESSING / COMPLETED / FAILED

class MultimediaAsset:
    def __init__(self, asset_id, filename: FileName, ...):
        self._status = AssetStatus.PENDING
    @property
    def status(self): return self._status
    def mark_as_failed(self, reason):
        if self._status is AssetStatus.COMPLETED:
            raise InvalidStateTransitionError(...)
        self._status = AssetStatus.FAILED
    def _transition(self, expected, to):
        if self._status is not expected:
            raise InvalidStateTransitionError(...)
        self._status = to
```

우선순위

먼저 손볼 세 가지: 모두 실제 버그와 이어집니다

1

FAILED 상태 추가

변환 실패 시 PROCESSING에 멈추는 문제를 막습니다.

2

저장 경로 규칙 한 곳으로

업로드와 다운로드에 중복된 규칙을 FileStorage로 모읍니다.

3

ID 경쟁 상태 해결

저장소가 ID를 처음에 발급해 동시 업로드 덮어쓰기를 막습니다.